复杂业务团队的 AI Coding 交付实践：知识库、RD 流程和质量门禁

原创 物流技术 大淘宝技术 2026年8月4日 23:28 浙江 90人



该文介绍了复杂业务场景下AI研发交付的实践方案，团队将流程分为三阶段，重点打造第一阶段底座。通过分层知识库设计（main全局业务、applications应用知识、candidate候选知识、personal个人经验、template模板），确保AI获取准确上下文；采用文件化的RD流程，用Markdown承载需求分析、拆解、实现校验全过程，使研发状态可接续、可review；前置质量门禁，在PRD验证、需求澄清、方案设计等关键节点设置人机协同review点，避免后期高成本返工。不追求100%全AI交付，强调AI负责分析实现、人聚焦关键判断，通过知识沉淀与流程规范提升整体交付质量与稳定性，为后续自动化和多Agent协同奠定基础。

01 整体背景

过去一段时间，复杂业务场景下的 AI 研发交付逐渐形成了一些共识：通过 Wiki 补齐团队上下文，通过 skills 和研发模板约束 AI 的分析、拆解和编码过程，再通过知识回补把需求里的经验沉淀下来。

在真正进入落地阶段后，每个团队都会遇到自己的具体问题，业务复杂度不同，应用数量不同，历史包袱不同，组织分工不同，发布和质量要求也不同。Wiki 怎么建，skills 怎么写，研发模板怎么拆，review 点放在哪里，最终都会有差异。

所以这篇文章主要讲我们团队是怎么设计的。

02 基础介绍

我们是一个复杂业务后端团队，系统中存在多个业务域、多个应用、多个模块，日常研发也会按应用、按模块、按业务方向协同。同时为了支撑各种繁杂的业务、保障稳定性、避免故障和资损，我们在架构上做了许多妥协，部分需求最终改动的代码可能不多，但前面的理解、拆

分层知识库设计

解、确认和 review 链路会比较复杂，代码不仅要会写，还要写在对的位置上，延迟系统腐化，所以某种程度上也比较依赖开发者的经验。

考虑到这种复杂性，也为了能够平稳落地，所以我们没有一开始就奔着"短时间全自动化"去做，而是把 AI 研发流程的落地分成了三个阶段。第一阶段先打底：建设知识库，沉淀团队上下文；再用 Coding Agent CLI + skills + Markdown 模板，把需求分析、应用拆解、实现校验和知识回补这条链路跑通。

我们内部把这套流程叫 RD，也就是 research development。

名字是我们取的，本质上它就是一组 skills、命令协议和 md 文档。当前主要通过 Coding Agent CLI 来执行，但它并不和某一个 Coding Agent 强绑定。只要其他 Agent 能读取这些 Markdown 文件，理解里面的 requirement、analysis、implementation-check 和 continue-prompt，也可以接着干活。

这也是我们最开始先做 Wiki + skills + RD 流程的原因。

通用 Coding Agent 会越来越强，模型能力也会快速变化，新产品也会不断出现。业务团队很难长期押注某一个工具形态。真正值得花时间打磨的，是那些最有团队特征、最难被通用工具直接替代的东西：业务知识、应用边界、研发规范、质量门禁、跨应用协作方式和历史经验。

工具可以换，模型可以升级，底层的团队上下文和研发协议需要自己沉淀。

我们这套实践可以概括成三步：

```
知识库 + 工具链
  -> 自动化流程
  -> 自主协同交付
```

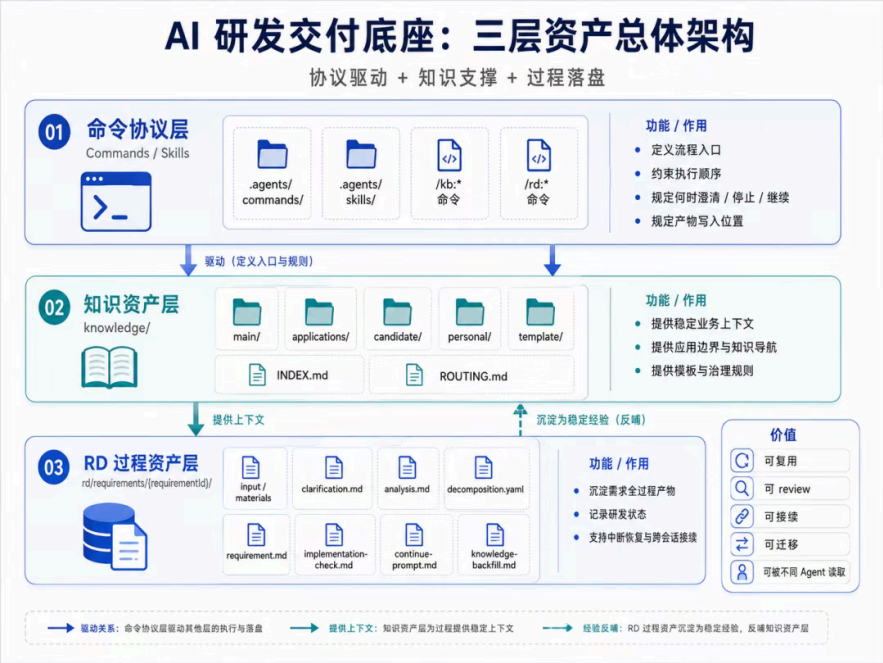
第一阶段先把知识库和 RD 流程跑稳。第二阶段再把更多流程动作交给 Agent 自动推进。第三阶段才是更完整的协同交付，包括开发、测试、发布、观测、回滚等生产链路。

这篇先讲第一阶段，也是我们认为最值得投入的底座部分，二三阶段我们正在开发中，后续也会有文章发出来分享。

03 总体设计

三层资产

分层知识库设计



当前这套系统可以理解成三层：

```
命令协议层
  -> .agents/commands/
  -> .agents/skills/

知识资产层
  -> knowledge/

RD 过程资产层
  -> rd/requirements/{requirementId}/
```

命令协议层定义 `/kb:*` 和 `/rd:*` 命令怎么工作。例如先读哪些规则文件，什么时候要澄清，什么时候必须停止，产物写到哪里，哪些事情可以继续执行，哪些事情需要人工确认。

知识资产层放在 `knowledge/` 下。这里沉淀正式知识、候选知识、个人经验、模板、路由规则。

RD 过程资产层放在 `rd/requirements/{requirementId}/` 下。这里保存某一个需求从输入、材料、澄清、分析、拆解、实现校验到知识回补的全过程产物，这些产物在设计上着重做了可以「跨会话执行」，解决上下文压缩导致的漂移问题。

用流程表示，大概是：

```
需求 / PRD / Bug / 变更
  -> /rd:verify-prd          - for产品，确保prd合格，前置输入质量
```

分层知识库设计

```
    -> /rd:work                    - 通用rd命令，会根据输入自动引导用户下一步
    -> /rd:clarify                 - 需求澄清，结合知识库对需求的不确认点进行补充
    -> /rd:analyze                 - 需求分析，对需求进行分析并产出模块的需求摘要
    -> /rd:decompose               - 需求拆解，生成by应用的requirement做为开发产物
    -> /rd:verify-requirement      - 对需求进行确认，确保提交给应用的需求没有遗漏
    -> 业务应用仓库开发
    -> /rd:apply                   - 使用requirement进行代码实现
    -> /rd:validate                - 对比需求和实际代码，更新实现状态
    -> /rd:code-review             - 结合原始需求，研发规范，对代码进行CR
    -> /rd:release-plan            - 生成发布计划，发布待确认点
    -> 发布
    -> 知识回补
```

这里的每个命令都不是为了多生成一个文档。它们更像研发过程里的检查点，把原来散落在聊天记录、人脑和临时文档里的判断，落到可以复用、可以 review、可以接续的文件里。

同时为了降低使用的难度，我们将 `/rd:work` 命令作为路由命令，其他命令作为原子能力。在使用 `/rd:work` 时，Coding Agent 会根据用户的输入和当前上下文，自动推断当前用户的意图，给出下一步引导，这样事实上大家只需要输入 `/rd:work` 一个命令即可。

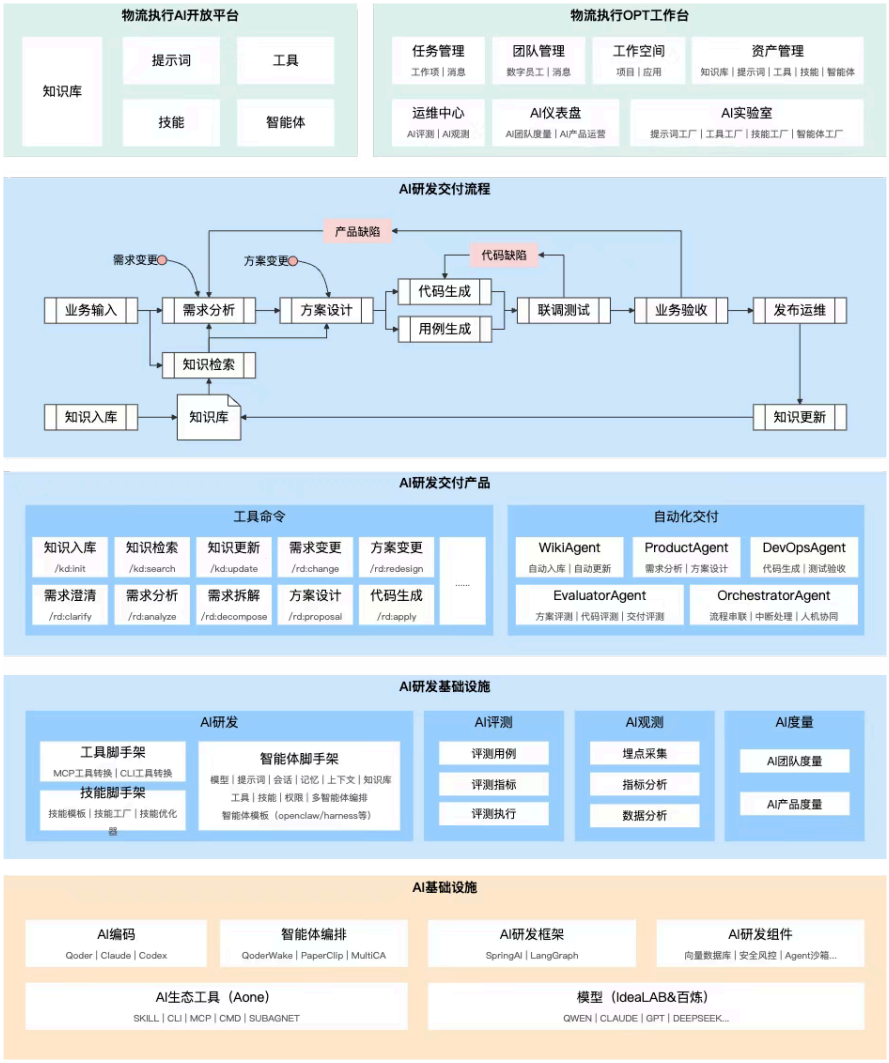
这套方法并不是为了替代所有研发流程，也不是说所有需求都要完整跑一遍 RD。毕竟对于非复杂应用、非复杂的业务，普通的提示词 + Coding Agent 就可以实现。

对于很小的局部修复、纯样式调整、一次性脚本、低风险工具需求，直接让 Agent 修改再人工 review，最后使用 kb 命令入一下知识库，可能就足够了。

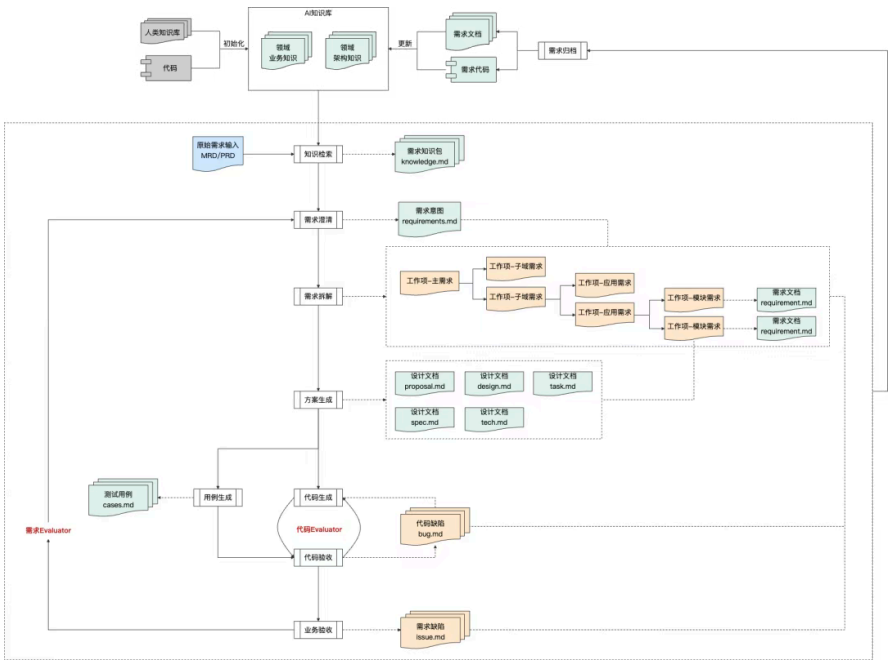
RD 更适合这类场景：需求跨多个应用，业务状态和上下游协议复杂，历史兼容逻辑多，发布风险高，或者需求本身需要多轮澄清和拆解，在这种场景下进行高质量的交付。

我们真正想解决的，是"复杂需求容易在理解、拆解和扩展点选择上走偏"的问题。

整体架构



研发流程



分层知识库设计

知识库设计：先分层，再进入研发流程

很多团队做 LLM Wiki 时，会先从"把文档整理给 AI 看"开始。这个方向没问题，但在复杂业务团队里，很快会遇到几个问题：

- 知识到底准不准；
- 哪些知识经过 owner 确认；
- 哪些只是历史材料里的线索；
- 哪些内容必须回到当前代码核对；
- 一个需求进来后，应该先读哪部分；
- 应用级知识和全局业务知识怎么区分；
- 个人经验、候选知识、正式知识怎么流转；
- 模板和目录规则怎么强约束。

如果这些问题不先设计清楚，Wiki 很容易变成另一堆"看起来有用、实际不知道怎么用"的文档。

所以我们把 `knowledge/`
设计成一个比较完整的知识资产目录，而不是只做应用文档集合。

当前核心目录是：

```
knowledge/
├── main/
├── applications/
├── candidate/
├── personal/
├── template/
├── INDEX.md
├── README.md
├── KNOWLEDGE-RULES.md
└── ROUTING.md
```

main：沉淀业务域内的通用知识

`knowledge/main/` 放的是跨应用、跨系统、跨业务线的通用知识。

例如：

- 业务域内的核心术语；
- 跨应用流程；

分层知识库设计

- 通用状态定义；
- 全局技术约束；
- 多应用都要遵守的业务规则。

这类知识不能归到单个应用里。如果强行放到某个应用目录下，其他应用在检索时容易漏掉，也容易把一个全局概念误解成某个应用自己的实现。

`main/` 更像整个业务领域的"公共语境"。

它不负责解释某个应用内部怎么写代码，而是帮 AI 先建立大方向：这个需求属于哪个业务域，可能涉及哪些角色，哪些概念是全局统一的，哪些流程可能跨应用。

applications：应用范围内的知识

`knowledge/applications/` 放每个应用自己的知识。

一个应用目录大致长这样：

```
knowledge/applications/application-xxx/
├── application-xxx.md
├── INDEX.md
├── domain/
│   ├── product/
│   ├── solution/
│   └── base/
└── tech/
```

这里面有几个设计点。

`application-xxx.md` 是应用总览。它说明这个应用负责什么，不负责什么，上下游是谁，核心模块有哪些，典型入口在哪里。

`INDEX.md` 是应用内导航。AI 进入某个应用后，不应该直接把整个目录读完，而是先读 INDEX，再按需求关键词、业务身份、Topic、接口、状态、模型等线索，决定下一步读哪些文件。

`domain/product/` 放产品能力知识。这里记录应用提供的通用能力，比如创单、取消、下发、回告、状态流转、幂等、路由、单元装配等。product 更关注主干流程，尽量写稳定能力。

`domain/solution/` 放解决方案知识。这里记录某个业务身份、某条业务线、某种履约模式对 product 能力的差异化扩展。solution 不应该把 product 全流程复制一遍，重点写差异：哪里扩展了，哪里覆盖了，哪里要兼容历史逻辑，哪里有特殊状态或规则。

`domain/base/` 放基础索引。比如 API、消息、模型、Repository、表、字段语义、调用关系。我们发现这类信息在需求分析和代码定位里非常高频，如果散落在各个 flow 文档里，AI 很难稳定命中。

`tech/` 放技术相关知识。例如研发规范、架构约束、框架使用方式、异常处理、调度任务、事务边界、MQ 处理规范、常见踩坑等。它不一定属于某个业务流程，但会直接影响代码实现质量。

一个示意性的应用目录结构（以下文件名仅作结构示例，不代表真实应用）：

```
knowledge/applications/application-core/
├── INDEX.md
├── application-core.md
├── domain/
│   ├── product/
│   │   ├── flow-create-order.md
│   │   ├── flow-cancel-order.md
│   │   ├── flow-dispatch-order.md
│   │   ├── flow-operation-report.md
│   │   ├── flow-update-address.md
│   │   ├── flow-order-control.md
│   │   ├── state-order-lifecycle.md
│   │   └── state-unit-lifecycle.md
│   ├── solution/
│   │   ├── solution-A/
│   │   │   ├── INDEX.md
│   │   │   ├── domain-overview.md
│   │   │   ├── flow-create-order.md
│   │   │   ├── flow-service-report.md
│   │   │   ├── rule-route-decision.md
│   │   │   └── state-service-unit.md
│   │   ├── solution-B/
│   │   ├── solution-C/
│   │   ├── solution-D/
│   │   └── legacy-adapter/
│   ├── base/
│   │   ├── api.md
│   │   ├── msg.md
│   │   ├── model.md
│   │   └── repository.md
│   └── README.md
└── tech/
    ├── tech-architecture-constraint.md
    ├── tech-framework-architecture.md
    ├── tech-process-routing.md
```

```
    ├── tech-product-extension.md
    ├── tech-error-mq-handling.md
    └── tech-scheduler-task.md
```

这个结构的好处是，AI 读应用知识时有明确路径：

```
先看应用职责
  -> 再看 product 主干能力
  -> 再看 solution 差异逻辑
  -> 再看 base 里的接口 / 消息 / 模型 / Repository
  -> 必要时读取 tech 里的研发规范和技术约束
  -> 最后回到当前代码确认
```

这里有一个很重要的原则：

> KB 提供稳定上下文，当前代码仍然是实现事实。

尤其是接口签名、DTO 字段、Topic 配置、feature key、状态枚举、开关配置这类容易变化的内容，知识库只提供定位入口。真正改代码前，还是要回到当前仓库核对。

candidate：候选知识暂存区

`knowledge/candidate/` 是候选知识区。

需求执行过程中，AI 会分析出很多有价值的信息，比如某个状态的业务含义、某个历史兼容逻辑、某个扩展点的使用方式、某条链路的上下游关系。

这些内容不应该全部直接写进正式知识库。

原因很简单：有些结论来自当前需求上下文，有些只是推断，有些还没有 owner 确认，有些可能只适用于一次需求。直接进入正式知识库，后面会误导 AI。

所以我们设计了 candidate 作为暂存区。

待合并知识先放这里，标清来源、证据、可信度和待确认项。后续经过 review，确认是稳定知识，再合并到 `main/` 或 `applications/` 里。

这能避免两个极端：

- 有价值的经验留在聊天记录里，下次没人找得到；
- 未确认的推断直接进入正式 KB，后面被 AI 当成事实引用。

personal：个人研发经验和踩坑记录

`knowledge/personal/` 放个人经验。

例如：

某类线上问题的排查经验；

某个模块的踩坑记录；

某种异常日志的判断方式；

某个历史坑的背景；

个人对某条链路的理解草稿。

这部分很重要。很多团队知识最开始都不可能天然是“正式文档”，它往往先存在于某个人的经验里。

但 personal 里的内容也不能直接代表团队结论。它更像个人工作区，可以帮助个人和 AI 快速复盘，也可以作为后续知识入库的素材来源。

如果某条 personal 经验被多次验证，或者被 owner 确认，就可以转成 candidate，再进入正式知识库。

template：强约束的知识写作模板

`knowledge/template/` 放各类文档模板。

例如：

- application 模板；
- domain 模板；
- flow 模板；
- state 模板；
- rule 模板；
- code 模板；
- tech 模板。

模板不是建议格式，而是强约束。

AI 写知识时，最容易出现的问题是粒度漂移：今天写成流程总结，明天写成代码笔记，后天写成一段散文。短期看都能读，长期维护会非常痛苦。

模板的作用是把知识的结构固定下来，下面是一个 application 模板的示例（占位符均为示意值）：

```
---
```

```yaml
# ==================== 必填字段 ====================
id: KB-APPLICATION-{DOMAIN}-{SEQ}       # 知识唯一编号
type: application                       # 知识对象类型：application
# 业务归属
domain: {domain}                        # 业务域
application: {appCode}                  # 应用编码
# 应用类型（供 AI 工具自动识别，如 code-review 据此触发端面安全检查）
appType: 后端应用                       # 取值：前端应用 / 后端应用
# 状态管理
status: DRAFT                           # 状态：DRAFT/CANDIDATE/OFFICIAL/D
sourceType: official                    # 来源：official/ai-assisted/perso
owner: {userId}                         # 负责人
version: 1                              # 版本号
updatedAt: YYYY-MM-DD HH:MM:SS          # 更新时间
confidence: medium                      # high/medium/low
stability: evolving                     # stable/evolving/volatile
evidence:
  - code: {核心模块或仓库路径}
  - doc: {应用文档或系统说明}
  - human: {确认人/时间}
# 标签与锚点
tags:                                   # 标签列表
  - {tag1}
  - {tag2}
anchors:                                # 锚点列表
  - APPLICATION:{appCode}
  - BIZ_IDENTITY:{identity1}
  - BIZ_IDENTITY:{identity2}
---
# {应用名称}
## AI 使用摘要
- 适用场景：需要了解 `{appCode}` 的系统职责、边界、上下游、核心模块时
- 关键入口：{核心接口/消息/启动模块}
- 关键规则：{本应用最重要的边界或约束}
- 关联知识：[INDEX.md](./INDEX.md)
- 使用前必须核对：应用代码路径、核心接口、消息 Topic 是否有近期变更
## 证据来源
| 类型 | 来源 | 说明 |
|------|------|------|
| code | {核心模块或仓库路径} | {代码核对说明} |
| doc | {应用文档或系统说明} | {文档来源说明} |
| human | {确认人/时间} | {人工确认说明} |
## 概述
（一句话描述这个应用是做什么的）
---
## 基本信息
```

```
| 属性 | 值 |
|------|-----|
| 应用编码 | {appCode} |
| 应用名称 | {应用名称} |
| 所属团队 | {团队名称} |
| 负责人 | {owner} |
| 技术栈 | {技术栈} |
---
## 系统职责
### 核心职责
（描述这个应用的核心职责）
### 业务范围
（描述负责的业务范围）
### 不负责什么
（明确边界，避免与其他系统混淆）
---
## 系统边界
（可以用文本图表示系统边界）
[上游系统A] ──┐
[上游系统B] ──┼──> [本应用] ──> [下游系统X]
[上游系统C] ──┘ |
 └──> [下游系统Y]
### 上游依赖
| 系统 | 依赖内容 | 调用方式 |
|------|----------|----------|
| {系统A} | （依赖什么） | RPC/MQ/HTTP |
### 下游被依赖
| 系统 | 提供什么 | 提供方式 |
|------|----------|----------|
| {系统X} | （提供什么） | RPC/MQ/HTTP |
---
## 核心模块
| 模块 | 职责 | 核心类 | 文档 |
|------|------|--------|------|
| {模块A} | （职责） | （核心类） | - |
---
## 变更历史
| 版本 | 日期 | 变更内容 | 变更人 |
|------|------|---------|--------|
| 1 | YYYY-MM-DD | 初始版本 | {owner} |
```

YAML Front Matter 里的字段让 AI 能判断：

- 这篇知识属于什么类型；
- 能不能作为事实引用；

- 证据来自代码、文档还是人工确认；
- 使用前是否要回到代码核对；
- 和哪些应用、状态、接口、Topic、模型、规则相关。

复杂业务里，错误知识比没有知识更危险。没有知识时，AI 可能会暴露不确定；错误知识被当成事实，就会直接把实现带偏。

ROUTING：让 AI 先定位，再读取



`knowledge/ROUTING.md` 是知识库里非常关键的入口。

收到一个需求后，AI 不应该全量读取 `knowledge/`。它应该先抽取：

- 关键词；
- 业务身份；
- 应用名；
- Topic；
- 接口名；
- 状态名；
- 模型名；
- 表名。

然后通过 ROUTING 定位：

分层知识库设计

```
候选业务域
  -> 候选应用
  -> 应用职责地图
  -> 知识入口
  -> 本地仓库路径
```

比如，需求里出现某个 Topic：

```
Topic
  -> ROUTING 定位目标应用
  -> 读取 application INDEX
  -> 读取 domain/base/msg.md
  -> 找到 Producer / Consumer
  -> 进入相关 product flow 或 solution flow
  -> 回到代码确认消费入口
```

需求里出现某个状态码：

```
状态码
  -> 查 state-*.md
  -> 查关联 flow-*.md
  -> 查相关 rule-*.md
  -> 定位代码入口
  -> 回到当前仓库核对实现
```

需求里出现某个业务身份：

```
业务身份
  -> 进入 main 里的领域定义
  -> 定位相关 applications
  -> 读取对应 solution 目录
  -> 对照 product 主干能力
  -> 分析差异实现
```

这个设计解决的是上下文窗口占用的问题。

复杂业务里的上下文不是塞的越多越好。更理想的方式是：AI 在正确阶段，读取正确粒度的上下文，只加载核心且对解决问题有用的那部分知识，避免被不重要的信息影响导致实现跑偏，所谓的渐进式加载。

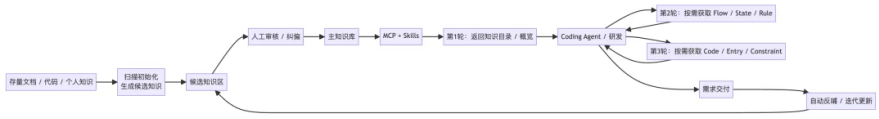
知识回补

对于知识库来说最怕的不是不完整，而是错误知识长期存在。

所以我们没有把 AI 总结出的内容直接写入 official，而是通过 candidate 承接。只有具备明确证据来源、经过 owner 或研发同学确认、并且被认为具备一定稳定性的内容，才会进入正式知识库。

对于接口签名、Topic、字段、枚举、开关这类变化较快的信息，即使进入知识库，也只作为定位入口，最终仍然要回到当前代码确认。

```
personal 个人经验
  -> candidate 候选知识
  -> owner review
  -> official 正式知识
  -> 需求执行中被引用
  -> 代码或业务变化后更新 / deprecated
```



05 RD 流程：用 Markdown 承载研发状态

知识库解决的是稳定上下文来源，RD 流程解决的是复杂需求怎么推进。

我们内部把这套 AI 研发流程简称为 RD 流程，表示从需求 research 到 development 承接的一套研发流程。它当前主要通过 Coding Agent CLI + skills 执行，但底层产物都是 Markdown 和 YAML，其他 Coding Agent 也可以读取。这点很关键。

同时我们没有把 RD 设计成某个工具里的私有状态。需求的所有关键过程全部落盘：

```
rd/requirements/{requirementId}/
├── source/
│   ├── input.md
```

```
4  │   ├── input.summary.md
5  │   ├── changes.md
6  │   ├── materials.yaml
7  │   └── materials/
8  ├── clarification.md
9  ├── execution-plan.md
10 ├── analysis.md
11 ├── analysis/
12 │   ├── application-a.md
13 │   └── application-b.md
14 ├── decomposition.yaml
15 ├── requirement-model.yaml
16 ├── status.md
17 ├── knowledge-backfill.md
18 └── applications/
19     ├── application-a/
20     │   ├── requirement.md
21     │   ├── implementation-check.md
22     │   └── continue-prompt.md
23     └── application-b/
24         └── requirement.md
```

这样做有几个好处：

第一，长会话可以拆开。大需求不需要在一个会话里从 PRD 一路写到代码。分析、澄清、拆解、实现、校验都可以分段完成。

第二，新会话可以接上。上下文不依赖聊天历史，而是依赖落盘文件。新开一个 Agent，只要读取 requirement、implementation-check、continue-prompt，就能知道当前做到哪里。

第三，人可以 review。每个阶段的关键判断都在文件里，不会藏在模型的中间推理里。人可以直接改 requirement，也可以在 review 时补充业务约束。

第四，其他工具可以接入。当前用某一款 Coding Agent CLI，后面也可以切到别的 Coding Agent。只要它能读这些 Markdown 文件，就能接入同一套研发协议。

1.原始需求不能直接进入编码

复杂需求进来后，RD 流程不会直接要求 AI 写代码。

第一步通常是 `/rd:verify-prd`。

它检查 PRD 或需求输入里有没有明显缺口，比如：

- 图片有没有文本化说明；
- 状态码是否明确；

分层知识库设计

- 上下游协议是否确认；
- open item 是否会阻塞开发；
- 非目标是否写清楚；
- 验收标准是否可验证；
- 是否缺少关键字段说明；
- 是否存在和知识库已有结论冲突的地方。

然后进入 `/rd:work`，保存原始输入和材料。

接着Coding Agent会提示你下一步应该做什么，并给出你命令和推荐的提示词，之后开发同学就可以通过 `/rd:clarify` 做澄清，通过 `/rd:analyze` 做知识检索和代码explore，通过 `/rd:decompose` 拆应用级 requirement。

进入编码前，再跑 `/rd:verify-requirement`。

- 当前应用的目标是什么；
- 非目标是什么；
- 影响哪些接口、消息、状态、字段、规则；
- 应该读哪些知识文件；
- 应该看哪些代码入口；
- 哪些校验要前置；
- 哪些逻辑是异步；
- 哪些能力要兼容历史；
- 哪些问题仍然 blocked；
- 验收标准是否可执行。

这两个 verify 命令体现了我们整个流程的设计哲学：fail-fast。

能在 PRD 阶段暴露的问题，不拖到 requirement。能在 requirement 阶段暴露的问题，不拖到编码。能在方案阶段暴露的问题，不拖到联调。越晚发现，返工成本越高。

2. requirement.md 是应用级开发契约

RD 流程里最关键的产物，是每个应用自己的 `requirement.md`。

它是 by 应用的开发契约。

一份合格的 `requirement.md` 至少要说清楚：

- 这个应用在本需求里的目标；
- 非目标和不能碰的边界；

分层知识库设计

- 相关知识依据和代码证据；
- 涉及哪些接口、消息、状态、字段和规则；
- 需要改哪些模块；
- 编码前优先阅读哪些入口；
- 需要满足哪些验收标准；
- 还有哪些问题没确认；
- 哪些稳定结论后续要回补知识库。

对于跨应用需求，每个应用都有自己的 requirement。这样编码会话不需要同时背着全部 PRD、全部技术方案、全部应用上下文，只聚焦当前应用的开发契约。

实际开发时，Agent 或研发同学按 requirement 一项项推进。做完就勾掉，做不下去就标 blocked，需求变化就更新 requirement。

这件事看起来朴素，但对复杂项目很有用。它把"靠聊天上下文推进需求"，变成了"靠文件化研发状态推进需求"。

3. validate 解决接续和对账问题

真实研发很少一次完成。

经常会有这些情况：

- 代码已经写了一部分；
- 需求中途变化；
- review 反馈插入；
- bugfix 改变了实现判断；
- 新会话不知道上一轮做了什么；
- AI 以为做完了，但 requirement 里还有遗漏。

所以 RD 里有 `/rd:validate`。

它会拿应用级 `requirement.md` 和当前本地分支 diff 做对账，判断每个需求项状态：

```
done：已经完成
partial：部分完成
todo：还没做
changed：需求或代码发生变化，需要更新 requirement
blocked：存在阻塞，需要人工确认
```

validate 之后会产出：

```
implementation-check.md
continue-prompt.md
```

`implementation-check.md` 记录 requirement 与代码实现的对账结果。`continue-prompt.md` 用于新会话继续开发。

这一步解决的是一个非常现实的问题：

> 开发可以中断，研发上下文不能丢。



我们这套实践里，最核心的两个点是知识库和质量门禁。

知识库决定 AI 能不能理解业务。质量门禁决定 AI 能不能进入真实交付。

很多人会把 AI Coding 想成这样：

```
把任务扔给 AI
  -> AI 自己写
  -> 自己测
  -> 自己修
  -> 不对就提示它重来一次
```

这个方向在很多局部任务上有效。但复杂业务里，"重来一次"的成本可能很高。

如果错的是某个局部实现，重新生成一次代码也许还行。如果错的是 PRD 理解、应用边界、扩展点选择、状态规则、上下游协议，重来就意味着整条链路倒回去：

```
重新澄清 PRD
  -> 重写 requirement
  -> 重做应用拆解
  -> 重做方案设计
  -> 重写代码
```

分层知识库设计

```
  -> 重跑测试
  -> 重新 review
```

所以我们更愿意把 review 点前置。

当前比较重要的门禁包括：

```
所以我们更愿意把 review 点前置。
当前比较重要的门禁包括：
/rd:verify-prd
  -> 输入质量门禁
clarify review
  -> 阻塞问题和业务口径确认
analysis / routing review
  -> 应用边界和影响范围确认
/rd:verify-requirement
  -> 编码前开发契约确认
方案 review
  -> 扩展点、架构路径、兼容策略确认
/rd:validate
  -> requirement 与代码 diff 对账
/rd:code-review
  -> 发布前代码质量和方案一致性确认
/rd:release-plan
  -> 发布、灰度、回滚、观测确认
knowledge backfill
  -> 稳定经验回补知识库
```

这里的人机 review 不是让人把 AI 做过的事情重做一遍。它更像交付系统里的关键检查点。人只在高价值位置介入，确认那些最容易造成返工和事故的业务事实。

例如：

- 某个校验应该前置还是后置；
- 某个状态从哪个 feature 里取；
- 某个历史服务是否要兼容；
- 某个异步事件是否允许真实调用外部服务；
- 某个方案是否影响已有链路；
- 某个字段是覆盖两份模型，还是只覆盖一份；
- 某个开关默认值怎么设；

分层知识库设计

- 发布时是否需要灰度和回滚。

这些判断，往往比写代码本身更关键。

我们也不追求 100% 全 AI。

如果 AI 做到 95%，剩下只是简单改两行，研发同学手改更快，那就手改。复杂业务交付的目标是总成本更低、质量更稳、风险更可控，不是追求"代码全部由 AI 生成"的纯度。



案例：一次跨阶段的真实交付

下面这个案例，是一次真实需求实践（以下涉及的应用名、类名、字段名、状态码均做了脱敏处理，仅作示意）。它能比较直观地说明：AI 交付质量的提升，靠的是知识库、requirement 和人工 review 点一起发挥作用。

需求大致背景：上游服务商通过一个新的回告状态触发"差异调整"，要求当前应用在主流程入口处做一系列前置校验，校验通过后落差异数据，并以事件驱动方式异步调用既有的订正流程。

1. 需求功能点

功能点
流程路由
数据合法性校验
业务规范校验
业务数据落库
业务事件发送
其他10+功能点脱敏处理

2. 第一轮：覆盖了大部分功能，但关键扩展点偏了

工具：Coding Agent CLI，模型选用最高能力档。

需求拆解输入：

```
/rd:work 需求文档：<内部需求链接，已脱敏>
```

需求澄清：

分层知识库设计

```
1、OQ-001：外部网关应用不需要改动
2、OQ-002：先走 xxx 主流程，然后基于服务更新事件驱动 xxx 对应流程
```

需求拆解结果里，领域拆分基本准确，需求文档也生成了，但漏掉了 `重量大于 0 的校验` 。

进入方案设计：

```
/rd:propose requirement.md 新开分支开发需求
```

我们主动要求 AI 自己 review：

```
需求和方案里有没有需要确认的点
```

AI 自己 review 出 7 个需要确认的点，进行了人工澄清：

```
1. xxx 读到的已经是覆盖后的值
2. xxx 通过事件监听器异步触发
3. xxx 先定义成一个内部枚举值
4. 新增字段
5. 两份重量模型都覆盖
6. 不幂等，但做最多调整次数限制，次数通过 内部平配置平台 配置
7. 用 order.feature[xxx]
```

但方案设计阶段仍然有两个明显丢失：

功能点	是否包含	丢失环节
业务规范性校验	[]	【design.md 阶段】误认为原有重量xx校验已符合要求
增加xxx记录	[]	【design.md 阶段】误认为 record 记录指的是 xxx 内部已有的记录

进入代码开发：

分层知识库设计

```
/rd:apply
```

第一轮 AI 代码分析结果：

功能点	是否覆盖	可用性	可用性说明
流程路由	是	100%	
数据合法性校验	是	100%	
业务规范性校验	是	50%	1、校验逻辑放在 xxx 了，导致无法前置阻断服务商回传，存在逻辑问题 2、遗漏了一项业务校验
业务数据落库	是	80%	多传递了一个不必要字段
发送处理事件	是	100%	
其余10+脱敏功能点	是	90%	1、核心业务逻辑实现无误 2、一项数据记录存在歧义理解有误，遗漏实现

第一轮结论：

```
AI 代码采纳率：约 75%（人工估计）
最大问题：
PRD、澄清及设计过程均未明确需要在 xxx 中执行前置校验逻辑。
```

这个结果很有代表性。

AI 不是完全不会做。它覆盖了大量功能点，主流程也能写出来，单测也能生成。

问题出在几个关键校验的位置被放到了 `xxx` 相对靠后的位置，无法前置阻断服务商回传。

这类问题不是简单的语法错误，也不是少写一个 if。它说明前面 PRD、澄清和方案阶段没有把关键架构约束表达清楚：

```
新回告状态进入
    -> xxx 前置校验
    -> 校验通过后记录xx更新数据
    -> 发出xx更新事件
    -> 异步调用 xxx
```

分层知识库设计

```
6     -> 复用订单订正逻辑
```

如果这个约束没有进入 requirement，AI 很容易理解"要做什么"，但把"在哪里做"理解错。

3. 第二轮：把关键约束写进 requirement，并加强 review

第二轮仍然使用同一个 Coding Agent CLI，模型也是最高能力档。

需求拆解输入：

```
/rd:work 需求文档：<内部需求链接，已脱敏>
```

澄清问题：

```
1、不涉及内部网关应用，需求中提到的"网关"指的是外部网关系统
2、变更来源枚举值定义：系统内部定义，包含服务商/运营两个枚举值
3、主子单场景下，xx重量从xx信息中取
```

人工 review 调整：

```
1. 针对新回告状态路由至 xxx，校验规则通过之后记录差异更新数据并发出差异更新事件
2. 监听差异更新事件异步调用 xxx，复用原有订单订正逻辑及事件，新增本次订正逻辑
```

这一次，关键变化是把"两阶段异步架构"明确写进了 requirement 和方案输入里。

方案设计输入（经 Coding Agent 自动重写后的版本）：

```
/rd:propose 基于提供的项目结构和需求文档，结合知识库仓库中相关业务的知识（先熟悉知识
```

这段 prompt 里最重要的是几个约束：

- 先读知识库下与本应用相关的知识；
- 先熟悉索引，再按需读取具体知识文件；

分层知识库设计

- 严格按 requirement 实现；
- 第一阶段在 `xxxProcess` 中处理新回告状态并执行六项前置校验；
- 第二阶段通过事件驱动方式在 `xxxProcess` 中完成订正。

随后继续人工 review：

```
1、运单存在性校验要注意兼容历史业务身份 B，其服务类型同样命中本次范围
2、运单状态校验要校验签收和拦截状态，从订单 xxx 的 xxx 中取
3、运单完结时间也应该取xx和xx对应的时间
```

这些信息很容易被遗漏。如果不显式补充，AI 很可能根据字面需求写出"看起来合理，但不符合历史业务兼容"的实现。

由于方案设计阶段窗口已经使用 85%，这里新开会话执行：

```
/rd:apply
```

第二轮 AI 代码分析结果：

功能点	是否覆盖	可用性	可用性说明
流程路由	是	100%	
数据合法性校验	是	100%	
业务规范校验	是	100%	AI推测计费重量为整数 未要求用户澄清
业务数据落库	是	80%	计费重量解析逻辑未考 虑主子单场景
发送事件	是	100%	
其他10+功能点脱敏处理	是	100%	

汇总结果：

```
1、中断次数：3（问题澄清 1 次、Review 调整 2 次）
   一定程度上依赖了第 1 轮沉淀的经验
2、代码采纳率：95% 以上
```

分层知识库设计



我们没有继续追求 100%。

到 95% 以后，剩下如果只是简单问题，手改更快就手改。在真实业务交付里，这样更经济。

这次案例给我们的结论比较直接：

第一轮不是 AI 完全做不了，而是关键业务约束没有在前置阶段被表达清楚。第二轮通过知识库、requirement 和 review 点，把关键约束放到了编码前，结果明显收敛。

我们没有只看"AI 写了多少代码"，而是更关注几个更贴近交付质量的指标：

- PRD 阶段发现了多少 open item；
- requirement 阶段拦截了多少不清晰项；
- validate 阶段发现了多少 requirement 与代码 diff 不一致；
- 代码采纳率大概是多少；
- 人工中断发生在哪些阶段；
- 问题是 PRD 表达不清、知识库缺失、扩展点判断错误，还是纯编码问题；
- 需求结束后回补了多少稳定知识。

这些指标比"生成了多少行代码"更能说明 AI 研发流程有没有真正变好。

08 为什么我们先打底，再自动化

做 AI 研发交付，很容易一上来就想做自动化：

```
1  自动接需求
```

分层知识库设计

```
自动拆任务
自动拉分支
自动写代码
自动跑测试
自动修复
自动发起发布
```

这些能力我们也会做，但不会把它们放在最前面。

因为自动化流程跑得越快，对底座的要求越高。如果知识库不准，requirement 不清楚，review 点没有设计好，自动化只会把错误更快地执行完。

所以第一阶段，我们更愿意把时间花在三件事上：

第一，知识库要打磨。让团队经验从 owner 脑子里、历史 PRD 里、聊天记录里、线上问题里，逐步变成可检索、可路由、可验证的知识资产。

第二，RD 流程要打磨。让需求输入、澄清、分析、拆解、实现校验和知识回补变成稳定协议，而不是每个同学、每个 Agent 都靠临场发挥。

第三，质量门禁要打磨。让人机 review 点放在最关键的位置，尽早发现高返工成本问题。

这个底座打稳以后，后面的自动化才有空间。

模型变强了，可以接进来。新的 AI Coding 产品出现了，也可以切过去。多 Agent 编排成熟了，也可以接到这些 Markdown 产物和命令协议上。

这也是我们把 RD 产物设计成文件的原因。工具可以变化，文件化的研发状态更容易迁移。



后面我们会继续把这套实践往 Harness 化推进。

这里说的 Harness，可以理解成一个确定性的研发交付环境。它会把知识库、工具链、权限、流程状态、质量门禁、验证规则和发布约束统一收进去，让 Agent 在团队定义好的轨道里工作。

第一层是运行环境的确定性。

Agent 进入一个需求时，应该知道：

- 本地应用仓库在哪里；
- 使用哪些命令；
- 先读哪些知识库入口；
- requirement 写在哪里；
- 实现校验怎么跑；
- 哪些文件是权威上下文；
- 哪些动作必须停下来等人确认。

第二层是研发协议的确定性。

Agent 不能拿到需求就直接写代码。它应该沿着团队定义好的路径推进：

分层知识库设计

```
输入先 verify
  -> 知识按 ROUTING 加载
  -> 需求过程分阶段落盘
  -> 应用级 requirement 先确认
  -> 编码按契约执行
  -> validate 对账
  -> 发布前 review
  -> 稳定知识回补
```

这个方向最终服务的不是某一次 AI 写代码写得多漂亮，而是团队交付能力的稳定提升。

每做完一个需求，团队都能多沉淀一点：

- 更完整的业务上下文；
- 更清楚的应用边界；
- 更稳定的研发流程；
- 更可复用的 requirement 模板；
- 更明确的 review 点；
- 更好的失败归因；
- 更多可以回补知识库的经验。

另外，在真实落地时我们也会控制 AI 访问边界：

只让 Agent 读取当前需求需要的仓库和知识目录；敏感文档、线上数据、密钥、客户信息不进入 prompt；对外分享时，需求名称、内部链接、应用名、类名和配置 key 都需要脱敏。

AI 研发流程越深入真实交付，权限和数据边界越要前置设计。



目前复杂业务下的 AI 研发交付，Wiki + skills + 研发模板已经逐渐成为共识。真正拉开差异的地方，在具体怎么设计、怎么治理、怎么进入团队研发流程。

我们的实践还在演进，但第一阶段已经有几个比较明确的判断。

第一，知识库要分层。`main` 放全局业务知识，`applications` 放应用范围知识，`candidate` 承接待确认知识，`personal` 保留个人经验，`template` 约束知识写作结构。应用内部再按 product、solution、base、tech 拆开，让 AI 能按需求逐步加载上下文。

第二，RD 流程要文件化。Coding Agent CLI + skills 只是当前执行方式，底层产物是 Markdown。只要 requirement、analysis、implementation-check、continue-prompt 这些文件稳定下来，后续切换 Coding Agent 或接入多 Agent 编排都会更顺。

第三，质量门禁要前置。PRD 要 verify，requirement 要 verify，方案要 review，实现要 validate，发布前要 code-review 和 release-plan。复杂业务里，晚发现的问题通常更贵。

第四，人机协同 review 是交付系统的一部分。人不需要在所有地方介入，但必须在关键业务事实、扩展点、兼容逻辑和发布风险上介入。AI 负责大部分分析和实现，人负责[illegible]终质量。

分层知识库设计

第五，不追求 100% 全 AI。做到 95%，剩下两行手改更快，那就手改。AI 研发交付的价值不在"纯度"，在总交付成本、质量稳定性和风险可控性。

这套东西看起来比"直接让 AI 写代码"慢一些。但在复杂业务团队里，前面多花一点时间把知识、流程和质量门禁设计好，后面才能少返工、少走偏、少靠 owner 临时救火。

AI 发展很快，工具和模型都会继续变化。团队自己的业务上下文、研发协议和质量门禁，越早沉淀，越容易跟上后面的变化。



本文作者寂秋，来自淘天集团–物流技术团队。一支专注于物流订单履约业务研发的技术团队。依托多元业务场景，我们持续探索与迭代技术能力，长期投入稳定性建设及智能化建设，为数百万商家提供稳定可靠的订单履约与物流体验，为数亿包裹安全高效流转保驾护航。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 1.3万

留言

写留言

还没有留言，来写首评

分层知识库设计